

NAMA : Muhammad Alfaras Aditya Rizky
KELAS : 2025B
NIM : 25091397065

Latihan Bab 8 — Queue (Antrian)
Struktur Data

Soal 1: Kompleksitas Waktu (Worst Case) — TicketCounterSimulation

Berikut adalah analisis kompleksitas waktu worst case untuk setiap operasi yang didefinisikan dalam class TicketCounterSimulation:

Metode/Operasi	Deskripsi	Kompleksitas Waktu (Worst Case)
<code>__init__()</code>	Inisialisasi objek simulasi; membuat Queue dan Array untuk agen, mengisi TicketAgent	$O(n)$ — $n = \text{numAgents}$
<code>run()</code>	Loop sebanyak <code>numMinutes+1</code> kali; tiap iterasi memanggil 3 handler	$O(m)$ — $m = \text{numMinutes}$
<code>printResults()</code>	Menghitung dan mencetak hasil; akses <code>len(queue)</code>	$O(1)$
<code>_handleArrival()</code>	Cek probabilitas kedatangan penumpang; enqueue jika tiba	$O(1)$
<code>_handleBeginService()</code>	Cek agent idle; dequeue penumpang & mulai layanan	$O(n)$ — $n = \text{numAgents}$
<code>_handleEndService()</code>	Cek agent selesai layanan; update statistik	$O(n)$ — $n = \text{numAgents}$

Penjelasan masing-masing operasi:

- `__init__()`: Kompleksitas $O(n)$ karena terdapat loop for sebanyak `numAgents` (n) untuk membuat dan menyimpan setiap objek TicketAgent ke dalam Array.
- `run()`: Kompleksitas $O(m)$ di mana $m = \text{numMinutes}$. Loop berjalan sebanyak $m+1$ kali dan tiap iterasi memanggil tiga handler. Jika handler di dalamnya $O(n)$, total menjadi $O(m \times n)$, namun karena jumlah agen biasanya konstan, kompleksitas dominan tetap $O(m)$.
- `printResults()`: Kompleksitas $O(1)$. Hanya melakukan operasi aritmatika sederhana dan memanggil `len()` yang berjalan dalam waktu konstan.

- d) `_handleArrival()`: Kompleksitas $O(1)$. Hanya membangkitkan bilangan acak dan melakukan satu operasi `enqueue`.
- e) `_handleBeginService()`: Kompleksitas $O(n)$. Mengiterasi seluruh array `theAgents` sebanyak n kali untuk mencari agen yang bebas.
- f) `_handleEndService()`: Kompleksitas $O(n)$. Sama seperti `_handleBeginService`, mengiterasi seluruh array `theAgents` sebanyak n kali.

Soal 2: Eksekusi Manual — Enqueue jika $i \% 3 == 0$

Kode yang dieksekusi:

```
values = Queue()
for i in range(16):
    if i % 3 == 0:
        values.enqueue(i)
```

Trace eksekusi manual untuk setiap nilai i dari 0 hingga 15:

i	$i \% 3 == 0?$	Operasi	Isi Queue (Front → Rear)
0	Ya	<code>enqueue(0)</code>	[0]
1	Tidak	-	[0]
2	Tidak	-	[0]
3	Ya	<code>enqueue(3)</code>	[0, 3]
4	Tidak	-	[0, 3]
5	Tidak	-	[0, 3]
6	Ya	<code>enqueue(6)</code>	[0, 3, 6]
7	Tidak	-	[0, 3, 6]
8	Tidak	-	[0, 3, 6]
9	Ya	<code>enqueue(9)</code>	[0, 3, 6, 9]
10	Tidak	-	[0, 3, 6, 9]

11	Tidak	-	[0, 3, 6, 9]
12	Ya	enqueue(12)	[0, 3, 6, 9, 12]
13	Tidak	-	[0, 3, 6, 9, 12]
14	Tidak	-	[0, 3, 6, 9, 12]
15	Ya	enqueue(15)	[0, 3, 6, 9, 12, 15]

Nilai i yang memenuhi kondisi $i \% 3 == 0$ dalam `range(16)` adalah: 0, 3, 6, 9, 12, dan 15. Nilai-nilai tersebut dimasukkan ke dalam queue secara berurutan.

Hasil akhir isi queue (Front → Rear):

[0, 3, 6, 9, 12, 15]

Soal 3: Eksekusi Manual — Enqueue dan Dequeue

Kode yang dieksekusi:

```
values = Queue()
for i in range(16):
    if i % 3 == 0:
        values.enqueue(i)
    elif i % 4 == 0:
        values.dequeue()
```

Catatan penting: Karena menggunakan `elif`, jika suatu nilai i memenuhi kondisi `if (i % 3 == 0)`, maka kondisi `elif` tidak diperiksa. Misalnya $i=0$ memenuhi keduanya ($0 \% 3 == 0$ dan $0 \% 4 == 0$), tetapi hanya `enqueue` yang dijalankan. Begitu pula $i=12$.

i	Kondisi	Operasi	Isi Queue (Front → Rear)
0	$i \% 3 == 0$	enqueue(0)	[0]
1	-	-	[0]
2	-	-	[0]
3	$i \% 3 == 0$	enqueue(3)	[0, 3]
4	$i \% 4 == 0$	dequeue() = 0	[3]

5	-	-	[3]
6	$i \% 3 == 0$	enqueue(6)	[3, 6]
7	-	-	[3, 6]
8	$i \% 4 == 0$	dequeue() = 3	[6]
9	$i \% 3 == 0$	enqueue(9)	[6, 9]
10	-	-	[6, 9]
11	-	-	[6, 9]
12	$i \% 3 == 0$	enqueue(12)	[6, 9, 12]
13	-	-	[6, 9, 12]
14	-	-	[6, 9, 12]
15	$i \% 3 == 0$	enqueue(15)	[6, 9, 12, 15]

Selama eksekusi, terjadi dua kali dequeue: pertama pada $i=4$ (mengeluarkan nilai 0) dan kedua pada $i=8$ (mengeluarkan nilai 3). Nilai yang tersisa dalam queue adalah hasil dari enqueue pada $i=6, 9, 12$, dan 15.

Hasil akhir isi queue (Front → Rear):

[6, 9, 12, 15]

Soal 4: Implementasi Metode yang Tersisa

Terdapat tiga metode yang belum diimplementasikan dalam class TicketCounterSimulation, yaitu `_handleArrival`, `_handleBeginService`, dan `_handleEndService`. Ketiga metode ini menangani tiga aturan utama simulasi.

people.py — Class TicketAgent dan Passenger import
random

```

class Passenger:
    def __init__(self, arrivalTime):
        self._arrivalTime = arrivalTime

    def arrivalTime(self):
        return self._arrivalTime

class TicketAgent:
    def __init__(self, idNum):
        self._idNum = idNum
        self._passenger = None
        self._stopTime = -1

    def isFree(self):
        return self._passenger is None

    def isFinished(self, curTime):
        return self._passenger is not None and self._stopTime == curTime

    def startService(self, passenger, stopTime):
        self._passenger = passenger
        self._stopTime = stopTime

    def stopService(self):
        thePassenger = self._passenger
        self._passenger = None
        return thePassenger

```

ticketsim.py — Implementasi Lengkap TicketCounterSimulation

```

import random

from array import Array
from llistqueue import Queue
from people import TicketAgent, Passenger

class TicketCounterSimulation:
    def __init__(self, numAgents, numMinutes, betweenTime, serviceTime):
        self._arriveProb = 1.0 / betweenTime
        self._serviceTime = serviceTime
        self._numMinutes = numMinutes
        self._passengerQ = Queue()
        self._theAgents = Array(numAgents)
        for i in range(numAgents):
            self._theAgents[i] = TicketAgent(i + 1)

```

```

self._totalWaitTime = 0
self._numPassengers = 0

def run(self):
    for curTime in
range(self._numMinutes + 1):
        self._handleArrival(curTime)
self._handleBeginService(curTime)
self._handleEndService(curTime)

def printResults(self):
    numServed = self._numPassengers - len(self._passengerQ)
    avgWait = float(self._totalWaitTime) / numServed
    print("
    print('Number of passengers served =', numServed)
    print('Number of passengers remaining in line = %d'
        % len(self._passengerQ))
    print('The average wait
time was %4.2f minutes.' % avgWait)

# Aturan simulasi #1: Penanganan kedatangan penumpang
def _handleArrival(self, curTime):
    if random.random()
<= self._arriveProb:
        passenger = Passenger(curTime)
self._passengerQ.enqueue(passenger)
self._numPassengers += 1

# Aturan simulasi #2: Mulai layanan ke penumpang
def _handleBeginService(self, curTime):
    for i in
range(len(self._theAgents)):
        if
self._theAgents[i].isFree():
            if not
self._passengerQ.isEmpty():
                passenger = self._passengerQ.dequeue()
                waitTime = curTime - passenger.arrivalTime()
self._totalWaitTime += waitTime
                stopTime =
curTime + self._serviceTime
self._theAgents[i].startService(passenger, stopTime)

# Aturan simulasi #3: Selesai layanan
def
_handleEndService(self, curTime):
    for i in
range(len(self._theAgents)):
        if
self._theAgents[i].isFinished(curTime):
            self._theAgents[i].stopService()

```

Penjelasan logika ketiga metode:

a) `_handleArrival()`: Setiap satuan waktu, dibangkitkan bilangan acak antara 0 dan 1. Jika nilai tersebut lebih kecil atau sama dengan `arriveProb` ($= 1/\text{betweenTime}$), maka satu penumpang baru tiba dan dimasukkan ke dalam antrian, serta penghitung `numPassengers` bertambah satu.

- b) `_handleBeginService()`: Iterasi seluruh agen. Jika ditemukan agen yang bebas (`isFree() = True`) dan antrian tidak kosong, penumpang di depan antrian di-dequeue, waktu tunggu dihitung (`curTime - arrivalTime`), lalu agen mulai melayani penumpang tersebut hingga `stopTime = curTime + serviceTime`.
- c) `_handleEndService()`: Iterasi seluruh agen. Jika sebuah agen telah selesai melayani (`isFinished(curTime) = True`), layanan dihentikan dengan memanggil `stopService()` sehingga agen menjadi bebas kembali.

Soal 5: Modifikasi ke Satuan Detik

Modifikasi class `TicketCounterSimulation` dari satuan menit ke satuan detik dilakukan dengan mengganti parameter `numMinutes` menjadi `numSeconds`, serta menyesuaikan nilai `betweenTime` dan `serviceTime` dalam satuan detik (1 menit = 60 detik).

Bagian kode yang diubah:

```
class TicketCounterSimulation:
    def __init__(self, numAgents, numSeconds, betweenTime, serviceTime):
        # arriveProb dihitung per detik (bukan per menit)
        self._arriveProb = 1.0 / betweenTime
        self._serviceTime = serviceTime # dalam detik
        self._numSeconds = numSeconds # ganti numMinutes # ... (sisanya inisialisasi sama)

    def run(self):
        for curTime in range(self._numSeconds + 1): # loop per detik
            self._handleArrival(curTime)
            self._handleBeginService(curTime)
            self._handleEndService(curTime)

    def printResults(self):
        numServed = self._numPassengers - len(self._passengerQ)
        avgWait = float(self._totalWaitTime) / numServed
        print('The average wait time was %4.2f seconds.' % avgWait)
```

Tabel 5.1 berikut menyajikan hasil eksperimen simulasi dalam satuan detik. Parameter `betweenTime`=120 detik (setara 2 menit) dan `serviceTime` berupa 180 atau 240 detik (setara 3 atau 4 menit):

Num Seconds	Num Agents	Avg Service	Time Between	Avg Wait	Passengers Served	Passengers Remaining
6.000	2	180	120	149,4	2.950	1

30.000	2	180	120	234,6	14.400	0
60.000	2	180	120	655,8	29.400	8
300.000	2	180	120	945,0	147.540	11
600.000	2	180	120	1.270,2	295.800	14
6.000	2	240	120	636,0	2.400	7
30.000	2	240	120	2.999,4	12.000	25
60.000	2	240	120	5.743,2	24.000	62
6.000	3	240	120	30,6	3.060	0
30.000	3	240	120	30,0	14.400	0
60.000	3	240	120	63,6	30.060	2

Tabel 5.1: Hasil eksperimen simulasi antrian loket tiket dalam satuan detik.

Catatan: Nilai dalam tabel merupakan estimasi berdasarkan konversi dari Tabel 8.1 buku teks (1 menit = 60 detik). Nilai aktual dari setiap eksekusi akan berbeda-beda karena simulasi bersifat probabilistik (Monte Carlo).

Soal 6: Fungsi Membalik Urutan Item dalam Queue

Dirancang sebuah fungsi `reverseQueue` yang membalik urutan seluruh item dalam queue. Fungsi ini hanya diperbolehkan menggunakan operasi Queue ADT (`enqueue`, `dequeue`, `isEmpty`), dengan memanfaatkan struktur data stack (list Python) sebagai perantara.

Implementasi fungsi:

```
def reverseQueue(queue):
    """
    Membalik urutan item dalam queue.
    Hanya menggunakan operasi Queue ADT.
    Menggunakan stack (list Python) sebagai struktur data bantu.
    """
    # Langkah 1: Pindahkan semua item dari queue ke stack
    stack = [] while not queue.isEmpty(): item =
```



```

queue.dequeue()    stack.append(item)    # push ke
stack

    # Langkah 2: Pindahkan kembali dari stack ke queue
    # Stack bersifat LIFO sehingga urutan menjadi terbalik
    while len(stack) > 0:    item = stack.pop()    # pop
    dari stack    queue.enqueue(item)

    return queue

```

Contoh penggunaan:

```

q = Queue() for val in [10,
20, 30, 40, 50]:
q.enqueue(val)
# Queue awal : Front -> [ 10, 20, 30, 40, 50 ] <- Rear

reverseQueue(q)
# Queue akhir : Front -> [ 50, 40, 30, 20, 10 ] <- Rear

```

Trace eksekusi manual:

Queue awal: Front $\rightarrow$ [10, 20, 30, 40, 50] $\leftarrow$ Rear

Langkah 1 — Pindahkan semua item dari queue ke stack (dequeue berulang):

```

dequeue  $\rightarrow$  10, stack = [ 10 ]    dequeue
 $\rightarrow$  20, stack = [ 10, 20 ]    dequeue  $\rightarrow$  30,
stack = [ 10, 20, 30 ]    dequeue  $\rightarrow$  40, stack
= [ 10, 20, 30, 40 ]    dequeue  $\rightarrow$  50, stack = [
10, 20, 30, 40, 50 ]

```

Langkah 2 — Pindahkan kembali dari stack ke queue (pop + enqueue):

```

pop  $\rightarrow$  50, enqueue(50), queue = [ 50 ]    pop  $\rightarrow$  40,
enqueue(40), queue = [ 50, 40 ]    pop  $\rightarrow$  30,
enqueue(30), queue = [ 50, 40, 30 ]    pop  $\rightarrow$  20,
enqueue(20), queue = [ 50, 40, 30, 20 ]    pop  $\rightarrow$  10,
enqueue(10), queue = [ 50, 40, 30, 20, 10 ]

```

Queue akhir: Front $\rightarrow$ [50, 40, 30, 20, 10] $\leftarrow$ Rear

Analisis kompleksitas:

- a) Kompleksitas waktu: $O(n)$ — setiap item dipindahkan tepat dua kali (queue ke stack, kemudian stack ke queue).
- b) Kompleksitas ruang: $O(n)$ — stack sementara membutuhkan ruang sebanding dengan jumlah item n dalam queue.